

SHAH INVESTOR'S HOME LIMITED

Synapse

Technical Architecture

Document	Technical Architecture and Design
System	Synapse — centralised access and reporting platform
Version	1.0
Status	Current as built
Audience	Engineers, architects, DBAs, operations
Prerequisite	Business Requirements Specification v1.0

Contents

1. Architectural Principles
2. Technology Stack
3. System Structure
 - 3.1 Layers
 - 3.2 Request lifecycle
 - 3.3 Source layout
4. Data Model
5. Authorisation Model
6. Dynamic Report and Form Engine
 - 6.1 Configuration entities
 - 6.2 Query binding
 - 6.3 Report mode
 - 6.4 Form mode
 - 6.5 Drill-down
7. Front-End Architecture
8. Security Design
9. Performance and Scalability
10. Data Retention and Housekeeping
11. Theming and Internationalisation
12. Build, Environment and Deployment
13. Operational Runbook
14. Extension Guide
15. Known Constraints

1. Architectural Principles

Five decisions shape the rest of this document. Each was taken deliberately and should not be reversed without understanding the consequence.

#	Principle	Consequence
P1	Derive authorisation, never duplicate it. Application access follows from menu permissions rather than being stored separately.	No second permission list can contradict the first. One code path answers both "which menus" and "which applications".
P2	Configuration over code. Reports and forms are data, not source files.	Adding a report needs no release. The cost is that query text is stored in the database and must be treated as trusted, administrator-authored content.
P3	Trust structure, never trust values. Query text comes from administrators; every user-supplied value is bound as a parameter.	The engine can execute arbitrary administrator SQL without exposing an injection surface to end users.
P4	Enforce data rules in the query, not in the caller. Ownership filtering is applied by the engine.	A report author cannot leak another user's records by forgetting a condition.
P5	Prefer query-time filters to background jobs. Scheduling and retention are expressed as predicates.	No scheduler to operate or monitor; correctness does not depend on a job having run.

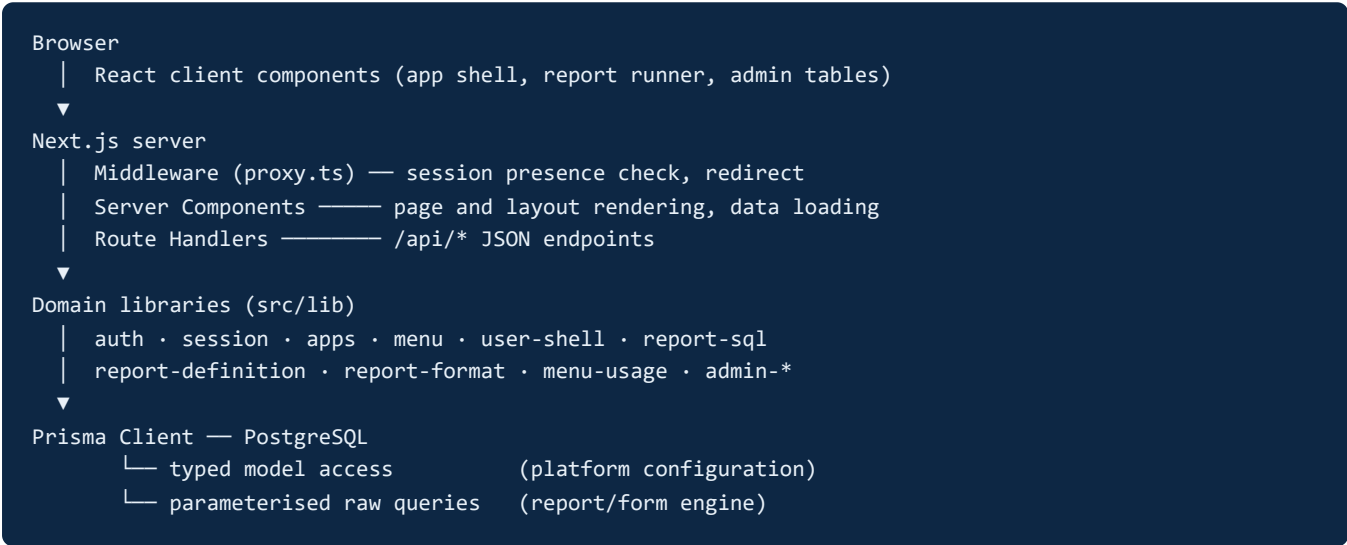
2. Technology Stack

Layer	Technology	Notes
Framework	Next.js 16, App Router, Turbopack	React Server Components by default; client components only where interactivity requires
Language	TypeScript (strict)	
Styling	Tailwind CSS v4	CSS-first configuration; design tokens declared in <code>globals.css</code> , no <code>tailwind.config.ts</code>
Database	PostgreSQL 16	Holds both platform configuration and reportable business data
Data access	Prisma 7 with driver adapters	<code>@prisma/adapters</code> over <code>pg</code> ; client generated to <code>generated/prisma</code>
Authentication	<code>jose</code> (JWT) + <code>bcryptjs</code>	Signed token in an httpOnly cookie

Validation	zod	One schema per administered entity, shared by create and update paths
Charting	recharts	Dashboard trend chart
Package manager	npm	

3. System Structure

3.1 Layers



3.2 Request lifecycle

1. **Middleware** verifies that a session cookie exists and is validly signed, and redirects unauthenticated requests to the login page. It performs no database access.
2. **The application layout** resolves the user's shell context — accessible applications, the active application, and the menu tree scoped to it — in a single pass.
3. **The page** loads only what is specific to it. A report page loads its report definition and breadcrumb.
4. **Client components** hydrate and take over interaction: filter entry, sorting, drill-down, administration dialogs.
5. **Route handlers** serve subsequent data operations — running a report, saving a record, listing notifications.

3.3 Source layout

Path	Contents
src/app/{app}/	Authenticated area: shared layout, dashboard, report route, twelve administration pages, and the loading boundary

src/app/login/	Login page, branding panel and assistance dialog
src/app/api/	Route handlers: auth, admin CRUD, reports, favourites, notifications, apps, menu usage
src/components/app-shell/	Top bar, sidebar, application switcher, notification bell, and the shell context providers
src/components/admin/	The generic administration table and its nested editor dialogs
src/components/reports/	Report runner, dynamic filter field, results table, drill-down dialog
src/lib/	Domain logic, shared by server components and route handlers
prisma/	Schema, hand-authored migrations, seed script

4. Data Model

All tables and columns are mapped to snake_case names. Entities fall into four groups.

Identity and organisation

Entity	Key	Purpose and notable fields
CompanyMaster	companyCode	Legal entity; PAN, TAN, registered address, logo path
AppMaster	appCode	One product within a company; icon, logo, display order
HierarchyMaster	hierarchyCode	Flat lookup of organisational positions with sequence
ClientCategoryMaster	clientCategoryCode	Client classification, default <code>c00</code>
UserDetails	uid	Credentials, contact, company, hierarchy, category, and lastAppCode
PasswordHistory	id	Prior password hashes for reuse checks
UserIpMapping	id	Permitted IP patterns; absence of an active row means unrestricted
LoginLog	id	Audit of every attempt with outcome and reason

Navigation and authorisation

Entity	Key	Purpose and notable fields
MenuMaster	menuCode	Self-referencing three-level tree. <code>menuType</code> is ROUTE, REPORT or EXTERNAL. <code>appCode</code> is nullable, where null means global.
RoleMaster	roleCode	Named permission set, scoped to company and hierarchy
RoleMenuMap	id	Role to menu, unique on role + menu + company + hierarchy
UserRoleMap	id	User to role
UserFavoriteMenu	id	Per-user favourites, unique on user + menu

Report and form engine

Entity	Key	Purpose and notable fields
FilterComponentMaster	componentCode	Reusable input: type, data source (static list, query, or none), option query
FilterDefinition	filterId	Named arrangement; collapsible and default-collapsed behaviour

FilterDefinitionItem	id	Placement of a component: row, position, mandatory, label override, default, mappedColumn for form mode, dependsOnItemId for cascading
ReportDefinition	reportId	Filter binding, mode, query text, target table, maximum rows, freeze columns, display style, footer note, permitted formats
ReportColumn	id	Display label and order, data type, decimals, highlight, total, identifier flag, and drill-down settings
ReportRowHighlightRule	id	Conditional row colouring by operator and comparison value

Communication and analytics

Entity	Key	Purpose and notable fields
Notification	id	Target type and target, delivery flags, message, optional schedule
NotificationRead	id	Per-recipient read state, unique on notification + user
MenuUsageLog	id	Denormalised selection record with no foreign keys; retained seven days

Why MenuUsageLog has no foreign keys. It is written on the click path, so inserts should not pay for constraint checks, and a log entry must remain meaningful after the menu or user it names has been deleted. The same reasoning applies to its denormalised company, application and menu names.

5. Authorisation Model

There is no user-to-application mapping table. Access is computed:

```

user
├─ active UserRoleMap (role must be active)
│   └─ RoleMenuMap (matched on user's company AND hierarchy)
│       └─ MenuMaster (menu must be active)
│           └─ appCode → AppMaster (must be active)

```

`getAccessibleMenuCodes()` is the single source of truth for that chain and is shared by both menu resolution and application resolution, so the two can never disagree. It is wrapped in React's `cache()`, which memoises it for the duration of one request — the layout and a report page both need it during a full render, and this collapses that to one resolution. The memoisation is strictly per request; a permission change takes effect on the next navigation.

Global menus. A menu with a null `appCode` appears in every application but grants access to none. Administration is configured this way deliberately: scoping it to a single application would allow an administrator to switch application and lose the ability to reach App Master and switch back.

Active application resolution. The user's stored `lastAppCode` is honoured only if they still have access to it; otherwise the first available application is used. A deactivated application or a withdrawn role therefore degrades gracefully rather than blocking sign-in. `lastAppCode` deliberately has no foreign key for this reason.

6. Dynamic Report and Form Engine

6.1 Configuration entities

A working report is three layers of configuration: reusable *components*, arranged into a *filter*, bound by a *report definition* to a query and display settings. Cascading is a property of the arrangement rather than the component, so the same component can be independent in one report and dependent in another.

6.2 Query binding

`src/lib/report-sql.ts` parses `:paramName` tokens in the stored query, produces a positional-parameter statement and an ordered value array, and executes it. Values are never interpolated into the statement text. Identifiers that must reach SQL — table names, mapped columns, the ownership column — pass through an identifier safety check.

A `DATE_RANGE` component binds as two parameters, `:CODE_FROM` and `:CODE_TO`.

6.3 Report mode

1. Mandatory filter values are validated, and date ranges checked for sane ordering.
2. A count is taken first. If it exceeds `maxRows`, no data is fetched and the client is told to offer an export instead.
3. Otherwise rows and column metadata are returned and rendered.
4. On success the applied filter values are written into the page URL, which is what allows a browser Back from a drill-down to restore the report exactly as it was.

6.4 Form mode

The same filter renders as a data-entry form. Each item's `mappedColumn` determines the column it writes. Ownership is enforced inside the statement, not by the caller:

```
-- listing the user's own records
SELECT * FROM (<queryText>) w WHERE w.created_by = $1

-- update and delete carry the same check
UPDATE <table> SET ... WHERE <id> = $1 AND created_by = $2 RETURNING *
DELETE FROM <table>      WHERE <id> = $1 AND created_by = $2 RETURNING <id>
```

Because the list query is wrapped, the configured query text must include `created_by` in its projection. Target tables must carry `created_by`, `updated_by` and `updated_on`; the engine populates them from the session on every write.

6.5 Drill-down

A column may open another report, either as a full page with a back link (PAGE) or in a sized popup (MODAL). Filter values carry across automatically: the entire current value set is forwarded and the target

ignores any key that is not one of its own component codes. This is why a summary and its detail report should reuse the same component codes for anything intended to travel.

Where a column's meaning depends on another filter in the same report — for example a single "Product / Region" column whose meaning depends on a grouping selection — the target parameter is stored as a discriminator map rather than a single code, and the current value of the named discriminator selects the destination parameter.

7. Front-End Architecture

The authenticated area is a shared layout wrapping a page slot. The layout renders the top bar and sidebar and establishes four context providers:

Provider	Supplies	Why it exists
LanguageProvider	Current language and the <code>t()</code> lookup	Interface strings only; administrator-authored content is never translated
ActiveAppProvider	Active application code	Lets client pages scope their own fetches without re-resolving permissions server-side
NotificationsProvider	Notification list and a read-marking function	The bell and the sign-in popup render the same underlying data; one fetch, one source of read state
MenuUsageProvider	Menu position index and <code>logSelection()</code>	Turns a click into a usage record using a map lookup, with no server round-trip to resolve position

Navigation behaviour. Menu items are client-side links; the top bar and sidebar are never remounted by navigation. A loading boundary under the authenticated area streams the incoming page, so the shell stays visible and interactive while only the content region changes. That boundary also enables link prefetching, which Next.js otherwise skips for dynamic routes.

Application switching writes the selection server-side, then navigates and refreshes. Because the menu tree is resolved in a server component, a refresh is what actually swaps it. The content region is keyed on the active application so client pages genuinely remount — without that, a page that loads data once on mount would continue showing the previous application's data.

8. Security Design

Concern	Control
Credential storage	bcrypt hashes only. Password history retained for reuse checks. Hashes are excluded from every export path.
Session	Signed JWT in an httpOnly cookie with a fixed lifetime. The signing secret is read at start-up and the process fails immediately if it is absent, rather than issuing unverifiable tokens.

Route protection	Middleware verifies the token signature before any authenticated route renders. Server-side helpers re-check the session; administration routes additionally require the administrator check.
Network restriction	Optional per-user IP patterns with wildcard octets, evaluated at login.
SQL injection	User values are always bound parameters. Query <i>text</i> is administrator-authored and reachable only behind the administrator check. Identifiers are validated before interpolation.
Record visibility	Ownership predicates are part of the statement, so they cannot be omitted by configuration.
Privilege escalation	Application selection is re-validated server-side against the user's derived entitlements before being stored.
Attribution	Usage and notification writes take identity from the session, never from the request body.

9. Performance and Scalability

9.1 Single-pass shell resolution

Composing the individual helpers to build the shell ran the permission chain twice and re-read the user record a third time. `getUserShellContext()` resolves applications, active application and the scoped menu tree in one pass. Measured on the reference dataset:

Approach	Time
Single-pass <code>getUserShellContext()</code>	18 ms
Composed helpers (previous)	42 ms

Both paths were verified to produce identical applications, active application and menu tree for every user before the change was adopted.

9.2 Indexing

PostgreSQL does not index foreign keys automatically. Indexes were added on the paths resolved on every request and on tables that grow without bound:

- `role_menu_map` (`company_code`, `hierarchy_code`, `role_code`) — the hottest path, evaluated on every page render — and (`menu_code`) .
- `menu_master` on `app_code` , `parent_menu_code` and (`company_code`, `hierarchy_code`) .
- Audit and history tables — `login_logs` , `user_ip_mapping` , `password_history` — on `user_uid` .
- Engine child tables — `report_column` , `report_row_highlight_rule` , `filter_definition_item` — on their parent key.
- `menu_usage_log` on `selected_at` and (`user_uid`, `selected_at`) , serving both the retention purge and the top-N read.

9.3 Other measures

Measure	Effect
Request-scoped memoisation of the permission chain	Layout and page share one resolution per render
Loading boundary under the authenticated area	Shell remains interactive during navigation; enables prefetch on dynamic routes
Single shared notifications fetch	Removes a duplicate request per page load and a second source of read state
Favourites and usage shortcuts take the application from the caller	Avoids re-running the permission chain in endpoints already fenced to the user's own rows

Fire-and-forget usage logging with <code>keepalive</code>	Survives the navigation it accompanies without delaying it
Row-count check before fetching report data	Prevents rendering oversized result sets
Server-side paging on User Master	The one administration list expected to grow large

Scaling watch-points. Administration lists other than User Master load in full and would need paging if any grows to thousands of rows — Menu Master is the most likely first. Report queries are executed synchronously; long-running analytical queries should be moved to the deferred export design rather than run interactively.

10. Data Retention and Housekeeping

Data	Retention	Mechanism
Menu usage log	7 days	Reads filter by the cutoff, so the window is correct regardless of housekeeping. A purge runs at most once an hour per process, triggered by a write.
Login log	Indefinite	Audit record; no automatic purge
Password history	Indefinite	Required for reuse checks
Notifications	Indefinite	Deactivate rather than delete to preserve the record

Scheduled notifications use the same principle: a future-dated message simply does not match the read query yet. There is no dispatch job, and therefore no failure mode where a job did not run.

11. Theming and Internationalisation

Three themes — day, night and hybrid — are expressed as CSS custom properties on the document root, with the palette taken from SIHL's public website. Alongside surface and text tokens there are semantic status tokens (`success` , `danger` , `warning` , `warning-surface` , `overlay`) and two chart colours, each defined per theme so that status colouring remains legible on dark backgrounds.

Charts read tokens at runtime. Recharts writes stroke as an SVG presentation attribute, where `var()` does not resolve. The dashboard therefore reads the chart tokens from the document and re-reads them when the theme attribute changes.

Ten languages are supported. Only interface chrome is translated; menu names, report titles and other administrator-authored content are stored once and shown as entered. Every new interface string must be added to all ten dictionaries.

12. Build, Environment and Deployment

Command	Purpose
<code>npm run dev</code>	Development server
<code>npm run build</code>	Production build; also the strongest pre-release check, catching server/client boundary violations that type-checking does not
<code>npm start</code>	Serve the production build
<code>npm run lint</code>	ESLint
<code>npx prisma db seed</code>	Reference data and the worked reporting example

Required environment variables:

```
DATABASE_URL    postgresql://user:password@host:5432/sihl_synapse
SESSION_SECRET  random string, 32+ characters
```

13. Operational Runbook

13.1 Applying a schema change

Never run `prisma migrate dev` or `prisma db push` against this database. It contains at least one hand-created table that is not a Prisma model, and both commands plan to drop it.

1. Edit `prisma/schema.prisma`
2. Hand-write `prisma/migrations/<timestamp>_<name>/migration.sql`
3. `npx prisma db execute --file prisma/migrations/<name>/migration.sql`
4. `npx prisma migrate resolve --applied <name>`
5. `npx prisma generate`
6. Restart the running server (see 13.2)

`prisma db execute` takes no `--schema` flag; it reads the Prisma config file. Prefer `ADD COLUMN IF NOT EXISTS` for columns on unmanaged tables so existing databases upgrade in place.

13.2 After regenerating the Prisma client

A running development server holds the previous client in memory. Stop the process, delete the `.next` directory and restart. Symptom of skipping this: new models appear missing at runtime while the code type-checks cleanly.

13.3 Verifying a change

- `npx tsc --noEmit` and `npm run lint` for correctness and style.
- `npm run build` to catch server/client boundary errors.
- Start the development server and confirm no CSS or compilation errors — the development CSS pipeline catches malformed stylesheets that the production build tolerates.

14. Extension Guide

14.1 Adding a report

1. **Components.** Create any filter components that do not already exist. Reuse existing ones wherever the meaning matches — this is what makes drill-down carry-over work.
2. **Filter.** Create a filter and arrange the components by row and position, marking mandatory items.
3. **Report.** Create the report definition with mode `REPORT`, the parameterised query, maximum rows and display settings.
4. **Columns.** Define each column with its label, order and data type; set decimals, totals and highlighting as needed.

5. **Menu.** Add a menu option of type REPORT pointing at the report, assign it to an application, and grant it to the appropriate roles.

14.2 Adding a data-entry form

As above, with mode FORM, a target table, and a `mappedColumn` on every filter item. Mark the primary key column as the identifier. The target table must carry `created_by`, `updated_by` and `updated_on`, and the list query must project `created_by`.

14.3 Adding an application

Create the application in App Master, then assign menus to it. Access follows automatically for anyone whose role grants one of those menus. Leave cross-cutting menus unassigned so they remain available in every application.

15. Known Constraints

Constraint	Impact
Report queries execute synchronously	Long-running queries hold a request open; the deferred export design exists but is not implemented
Export is CSV only	Excel and PDF output are future work
Administration lists other than User Master are unpaginated	Acceptable at current volumes; Menu Master is the first likely to need paging
Usage analytics limited to seven days	Trend analysis beyond a week is not possible without a retention change
Retention purge is per process	In a multi-instance deployment each instance purges independently; harmless but redundant
Dashboard statistics are indicative	Stat tiles and the trend chart are not yet wired to live data